به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین اوّل

مدرس: دکتر محمد قدسی، مهدی صفرنژاد — انتشار: ۶ اسفند ۹۹

---

# مقدمه و تحلیل الگوریتم‌ها

زمان تحویل این تمرین ۲۴ اسفند ۹۹ می‌باشد. سؤالات را با دقت بخوانید و روی همه آن‌ها وقت بگذارید. تمرین‌های تئوری تحویل گرفته نمی‌شوند اما از آن‌ها سؤالات کوییز مشخص می‌شود. بنابراین روی سؤالات به خوبی فکر کنید و در کلاس‌های حل تمرین مربوطه شرکت کنید.

---

سؤال ۱. فرض کنید یک آرایه $n$ عضوی به نام $A$ و یک عنصر $x \in A$ داریم که جایگاه دقیق آن را نمی‌دانیم. می‌خواهیم اندیس $i$ را پیدا کنیم، به طوریکه $A[i] = x$. با توجه به نکات ذکر شده به دو قسمت زیر پاسخ دهید.

آ. فرض کنید می‌دانیم احتمال اینکه $x$ در نیمه اول آرایه باشد، $p$ برابر این است که در نیمه دوم آرایه باشد($p \geq 1$). همچنین احتمال حضور آن در هر جایگاه برای هر دو نیمه یکسان است. پیچیدگی زمانی جستجوی خطی را در بدترین و بهترین حالت و حالت متوسط حساب کنید.

ب. فرض کنید که آرایه $A$ را به صورت مرتب شده به ما داده‌اند. پیچیدگی زمانی جستجوی دودویی را در بدترین و بهترین حالت و حالت متوسط حساب کنید. الگوریتم جستجوی دودویی در زیر آورده شده است.

---

$A \leftarrow sorted\ array$
$x \leftarrow value\ to\ be\ searched$
$binary_search(1, Array_Size)$
**procedure** $binary_search(low, up)$
  $mid = \frac{(low+up)}{2}$
  **if** $up < low$ **then**
    $return$ "$not\ found$"
  **if** $A[mid] < x$ **then**
    $return\ binary_search(mid + 1, up)$
  **if** $A[mid] > x$ **then**
    $return\ binary_search(low, mid - 1)$
  **if** $A[mid] = x$ **then**
    $return$ "$x\ is\ at\ location$" $mid$

---

پاسخ:

۱. در بهترین حالت، اولین خانه‌ای که بررسی می‌کنیم، همان خانه‌ای است که به دنبالش بودیم. پس پیچیدگی زمانی این حالت $O(1)$ است.

در بدترین حالت مجبور هستیم که تمام خانه‌ها را بررسی کنیم. پس در این حالت، پیچیدگی زمانی برابر با $O(|A|)$ است.

در حالت متوسط، اگر احتمال این که پاسخ در نیمه‌ی دوم باشد را q بنامیم، احتمال قرارداشتن پاسخ در نیمه‌ی اول pq است. حال اگر بدانیم پاسخ در یک بخش است، می‌دانیم که تمام خانه‌ها هم احتمال هستند. پس امیدریاضی تعداد خانه‌هایی که در این حالت باید طی کنیم، نصف خانه‌های آن نیمه است(چرا؟). پس می‌توان نوشت:

$$pq + q = 1 \rightarrow q = \frac{1}{p+1}$$

$$ExpectedValue = pq * (\frac{|A|}{4}) + q * (\frac{|A|}{2} + \frac{|A|}{4})$$

$$ExpectedValue = \frac{p}{p+1} * \frac{|A|}{4} + \frac{1}{p+1} * \frac{3|A|}{4} = \frac{|A|(3+p)}{4(p+1)} \leq \frac{|A|(3+1)}{4(0+1)} = |A|$$

هم‌چنین می‌توان گفت:

$$\frac{|A|(3+p)}{4(p+1)} \geq \frac{|A|(3+0)}{4(1+1)} = \frac{3A}{8}$$

پس اردر زمانی، همان $O(|A|)$ می‌ماند.

۲. بهترین حالت در این روش، مانند بهترین روش در قسمت قبل از $O(1)$ است.

بدترین حالت این روش، نیازمند $log(n)$ مرحله است.

به صورت میانگین، می‌توان هزینه‌های یافتن تمام خانه‌ها را حساب کنیم و سپس میانگین بگیریم.

$$\#Movements = 1 * 2^0 + 2 * 2^1 + 3 * 2^2 + ... + log(n) * 2^{log(n)-1}$$

یعنی نیمی از خانه‌ها هستند که حداقل به $log(n)$ مرحله نیاز دارند. پس میانگین از اردر $O(log(n))$ خواهد بود.

سؤال ۲. در یک آرایه با اندازه $n$، به عضوی که بیشتر از $\frac{n}{k}$ بار تکرار شده‌است، عضو اکثریت می‌گوییم. راهکاری ارائه دهید که با پیچیگی حافظه $O(k)$ عضو اکثریت را در صورت وجود پیدا کند.
[راهنمایی : راه حل برای $k = 2$]

پاسخ: برای حل این مسئله، $k - 1$ تا زوج مرتب تعریف می‌کنیم که مولفه‌ی اول هر زوج مرتب، نشان‌دهنده‌ی یک عدد است و مولفه‌ی دوم، نشان‌دهنده‌ی تکرر آن عدد است. فرض می‌کنیم که آرایه به شکل استریم ورودی گرفته می‌شود.

در هر گام، به شکل زیر عمل می‌کنیم. در ابتدا تمام زوج مرتب‌ها خالی است.

- یک خانه از آرایه را ورودی می‌گیریم.
- اگر یکی از زوج مرتب‌ها بود که مولفه‌ی اولش با مقدار خوانده شده یکی بود، تکرر آن را یکی زیاد می‌کنیم.
- اگر یکی از زوج مرتب‌ها بود که خالی بود، مقدار آن را برابر با مقدار خوانده شده قرار می‌دهیم و تکرر آن را یک می‌کنیم.
- در غیر این‌صورت، تکرر تمام زوج‌مرتب‌ها را یکی کم می‌کنیم. اگر تکرر یک زوج مرتب ۰ شد، فرض می‌کنیم که آن زوج مرتب خالی شده است.

حال ادعا می‌کنیم که اگر یک عضو اکثریت وجود داشته باشد، حتما مولفه‌ی اول یکی از زوج مرتب‌ها خواهد بود.

اثبات: فرض کنید که یک مقدار اکثریت وجود داشته باشد. هر بار که یک عدد در ورودی می‌خوانیم، یکی از دو حالت زیر اتفاق می‌افتد.

- تکرر یک زوج مرتب را افزایش می‌دهد.
- از تکرر تمام زوج‌مرتب ها یکی کم کند.

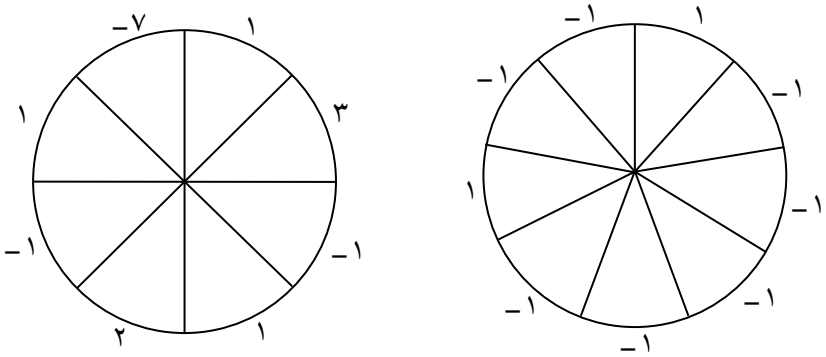
می‌دانیم که در مجموع حداکثر n بار تکرر یک زوج مرتب افزایش می‌یابد. چون اتفاق دوم، هر بار k تا از مجموع تکرر اعداد کم می‌کند، حداکثر $\frac{n}{k}$ بار اتفاق می‌افتد. حال فرض کنید که برای عضو اکثریت، a بار اتفاق اول افتاده باشد و b بار هم اتفاق دوم. می‌دانیم :

$$a + b > \frac{n}{k} \rightarrow a > \frac{n}{k} - b$$

پس تعداد دفعاتی که تکرر عدد اکثریت افزایش می‌یابد، بیشتر از تعداد دفعاتی است که تکررش کم می‌شود. بنابراین حتما این عضو در آخر کار، مولفه‌ی اول یک زوج مرتب است.

حال برای این که مطمئن شویم این عضو، یک عضو اکثریت است، کافی است یک بار دیگر تعداد تکرار های این عضو مشخص را در آرایه بشماریم(یعنی نیازمند این هستیم که آرایه دوبار استریم شود) و حافظه‌ی اضافی‌ای که گرفتیم هم از $O(k)$ است.

سؤال ۳. فرض کنید یک دایره به $n$ قطاع برابر تقسیم شده و روی هر کدام از قطاع‌ها یک عدد صحیح نوشته‌شده. می‌خوایم تعدادی قطاع انتخاب کنیم به طوری که فاصله‌های آن‌ها از هم برابر باشد (مثلاً در دایره با تعداد زوج قطاع یکی در میان انتخاب کنیم) و جمع اعداد روی قطاع‌های انتخاب شده حداکثر شود. به عنوان مثال در دو شکل زیر قطاع‌های انتخاب شده برای حداکثر کردن جمع، مشخص شده‌اند.




آ. یک الگوریتم با پیچیدگی زمانی $O(n \times h(n))$[1] بدهید که خواسته مسئله را انجام دهد.

ب. ثابت کنید که الگوریتم بدست آمده با پیچیدگی زمانی $O(n \times h(n))$ از $O(n \times \log n)$ است.

پ. ثابت کنید که الگوریتم بدست آمده با پیچیدگی زمانی $O(n \times h(n))$ از $\Omega(n \log n)$ و درنتیجه $\Theta(n \log n)$ است.

پاسخ:

۱. ابتدا فرض کنید که دایره را از یک قسمت برش داده‌ایم تا این آرایه‌ی دایروی به یک آرایه‌ی خطی تبدیل شود. حال پیدا کردن بخش‌هایی با فاصله‌ی یکسان هم‌ارز این است که سوال زیر را حل کنیم.

فرض کنید که q یک مقسوم علیه n است. اگر همه‌ی عضوهایی از آرایه‌ی خطی را انتخاب کنیم که باقیمانده‌ی اندیسشان بر q با هم یکسان باشد، یک حالت از انتخاب قطعه‌ها در آرایه‌ی دایروی با فاصله‌ی یکسان است و بالعکس. پس به کافی است به ازای هر q و هر باقی‌مانده‌ی ممکن، عضوهای با اندیس مناسب را پیدا کنیم و مجموع آن را حساب کنیم.

برای حل این سوال، به این شکل عمل می‌کنیم: ابتدا دایره را از یک قسمت دلخواه برش می‌دهیم تا آرایه‌ی خطی ساخته شود. سپس در $O(n)$ تمام مقسوم‌علیه‌های عدد n را پیدا می‌کنیم. حال کافی است به ازای هر مقسوم علیه n مانند q، آرایه را به q قسمت تقسیم کنیم. هر قسمت، اعضایی هستند که اندیسشان، باقی‌مانده‌ی یکسانی به q دارند. این کار هم به راحتی در $O(n+q) = O(n)$ انجام پذیر است.

سپس مجموع اعضای هر دسته را حساب کنیم. پاسخ مسئله، برابر است با ماکسیمم مقدار بدست آمده.

اردر زمانی این راه هم $O(n * \sigma(n))$ است که منظور از $\sigma(n)$ همان تعداد مقسوم‌علیه‌های عدد n است.

۲. حال می‌خواهیم ثابت کنیم که تعداد مقسوم‌علیه‌ها از اردر $\sqrt{n}$ است.

فرض کنید که n تعداد مقسوم علیه داشته باشد. این مقسوم‌علیه‌ها را $q_1, q_2, ..., q_k$ می‌نامیم. می‌دانیم که

$$q_i * \frac{n}{q_i} = n$$

بنابراین اگر $q_i$ مقسوم‌علیه n باشد، $\frac{n}{q_i}$ هم مقسوم‌علیه n می‌باشد. حال تعدادی دسته از روی مقسوم‌علیه‌های n می‌سازیم به صورتی که هر مقسوم علیه دقیقا در یک دسته ظاهر شود. این کار را به این شکل انجام می‌دهیم که اگر q مقسوم‌علیه n باشد،

[1] $h(n) = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + ... + \frac{1}{n}$

q را با $\frac{n}{q}$ در یک دسته قرار می‌دهیم. واضح است که در این صورت، هر مقسوم علیه تنها در یک دسته ظاهر می‌شود ولی می‌تواند در یک دسته دو بار تکرار شود. حال ادعا می‌کنیم که تعداد این دسته‌ها از اردر $\sqrt{n}$ است. زیرا بین اعضای هر دسته، حداقل یک عضو از $\sqrt{n}$ کوچکتر مساوی است. زیرا اگر هر دو عضو بزرگتر اکید از $\sqrt{n}$ باشند، ضربشان هم بزرگتر از n می‌شود که با قاعده‌ی دسته‌بندی ما در تناقض است. پس تعداد دسته ها از $\sqrt{n}$ کمتر مساوی است و تعداد کل مقسوم‌علیه‌ها، حداکثر برابر با دو برابر تعداد دسته‌هاست. پس تعداد کل مقسوم‌علیه‌ها حداکثر $2\sqrt{n}$ است و حکم ثابت شد.

سؤال ۴. توابع زیر را از لحاظ رشد مرتب کنید.[۲]

$$\begin{array}{lll} f_1(n) = \sqrt{lgn} & f_2(n) = lg(n^{lgn}) & f_3(n) = 100! \\ f_4(n) = n^{\frac{1}{lgn}} & f_5(n) = nlg(n) & f_6(n) = lg(n!) \\ f_7(n) = ln(n) & f_8(n) = lg^*(lg(n)) & f_9(n) = lg^*(n) \end{array}$$

پاسخ: در تابع‌های زیر، از چپ به راست، پیچیدگی افزایش می‌یابد:

$$f_3 = f_4 = O(1) < f_8 = f_9 < f_1 < f_7 < f_2 = O(lg(n)^2) < f_5 = f_6 = O(nlog(n))$$

سؤال ۵. با استفاده از درخت بازگشت، پیچیدگی تابع $T(n)$ را در هر قسمت به دست آورید.

آ. $T(n) = 2T(n/2) + \Theta(nlogn)$

ب. $T(n) = 3T(n/4) + 5n$

پ. $T(n) = 2T(n-1) + \Theta(n)$

پاسخ:

۱.

$$T(n) = 2T(\frac{n}{2}) + \Theta(nlogn)$$

برای حل این سوال، همان روش درختی را که برای تحلیل اردر

$$T(n) = 2T(\frac{n}{2}) + \Theta(n)$$

به کار بردیم را بنویسید. تنها تفاوتی که رخ می‌دهد، وجود تابع لگاریتم است که ابتدا با کران بالای $log(n)$ برای لگاریتم هر جمله، می‌توانید به اردر $O(nlog(n)^2)$ برسید. برای اثبات $\Omega(nlog(n)^2)$ کافی است که $\frac{log(n)}{2}$ مرحله‌ی اول را در نظر بگیرید و جای تابع لگاریتمی که در هر تابع ظاهر شده است، کران پایین $log(\frac{n}{2^{\frac{log(n)}{2}}}) = log(\sqrt{n}) = \frac{log(n)}{2}$ را قرار دهید و نتیجه بگیرید که اردر تابع، از $\Omega(nlog(n)^2)$ است. پس در کل تابع از $\Theta(nlog(n)^2)$ است.

[۲] عبارت $lg^*(n)$ به این معنی است که از n چند بار باید لگاریتم گرفت تا به عددی کمتر یا مساوی ۱ تبدیل شود.

۲.

$$T(n) = 3T(\frac{n}{4}) + 4n$$

$$T(n) = 4n + 3 * 4 * \frac{n}{4} + 3^2 * 4 * \frac{n}{16} + ... + 3^{log_4 n} * 4 * \frac{n}{4^{log_4(n)}}$$

$$T(n) = 4 \sum_{i=1}^{log_4(n)} (\frac{3^i}{4^i} n) < T(n) = 4 \sum_{i=1}^{\infty} (\frac{3^i}{4^i} n) = \Theta(n)$$

آخری تساوی‌ای که نوشته شده است، به کمک جمع جملات دنباله‌ی هندسی بدست آمده است.

۳.

$$T(n) = 2T(n-1) + \Theta(n)$$

$$T(n) = 2T(n-1) + c_1 * n = 4T(n-2) + 2c_2(n-1) + c_1 n = ... = \sum_{1}^{n} (c_i * 2^i * (n-i))$$

که چون جملات به شکل نمایی افزایش می‌یابند، اردر تابع مورد نظر، از اردر جمله‌ی آخر است که برابر است با $2^n$.

سؤال ۶. میخواهیم روی یک رشته $n$ بیتی از ۰ و ۱ الگوریتم زیر را اجرا کنیم. با توجه به این الگوریتم به سوالات زیر پاسخ دهید.

```
while true do
    cnt := 0
    for i = 1 → n − 3 do
        if s[i] = 1 and s[i + 1] = 0 and s[i + 2] = 1 then
            s[i] := 1 − s[i]
            s[i + 1] := 1 − s[i + 1]
            s[i + 2] := 1 − s[i + 2]
            cnt := cnt + 1
    if cnt = 0 then
        break
```

آ. ثابت کنید این الگوریتم به پایان می‌رسد.

ب. پیچیدگی زمانی الگوریتم را در بهترین و بدترین حالت حساب کرده و برای هر کدام یک مثال بزنید.

پاسخ: برای اتمام پایان پذیری این الگوریتم، می‌خواهیم از نزول متناهی استفاده کنیم. تعداد یک‌های رشته‌ی اولیه را در نظر بگیرید. هر بار که if اجرا شود، تعداد یک‌ها یکی کاهش می‌یابد. در نتیجه تعداد دفعاتی که if اجرا می‌شود محدود است. پس بالاخره در یک زمان، for اجرا می‌شود بدون این که if اجرا شود و الگوریتم در آن مرحله اتمام می‌پذیرد.

می دانیم در هر صورت حداقل یک بار for اجرا می‌شود. پس حداقل اردر زمانی $O(n)$ است. هم‌چنین اگر همه‌ی رشته از ۰ تشکیل شده باشد، تنها یک بار for اجرا می‌شود و در نتیجه الگوریتم از اردر $\Theta(n)$ خواهد بود.

اما ادعا می‌کنیم در بدترین حالت، این الگوریتم از اردر $\Theta(n^2)$ خواهد بود. برای این حالت، رشته‌ی زیر را در نظر بگیرید که از $n-1$ تا ۱ و ۱ عدد ۰ تشکیل شده است.

1111...1101

پس از اجرای هر بار for، تنها زیر رشته‌ی ۱۰۱ یکی به سمت چپ شیفت می‌خورد. بنابراین در اردر $\Theta(n)$ بار for اجرا می‌شود . هر بار اجرا هم از اردر $\Theta(n)$ طول می‌کشد. پس اردر کل الگوریتم از اردر $\Theta(n^2)$ خواهد شد.

موفق باشید